

ESTRUTURAS DE DADOS - PROVA 2

- 1) Considere o código-fonte parcial de uma tabela hash dada em anexo. Implemente as seguintes funções do arquivo **hash.c**:
- [1 ponto] **hash_table_set** para inserir/atualizar pares chave-valor. Lembre-se que na atualização, o valor anterior deve ser retornado para ser desalocado na main. Colisões devem ser resolvidas com inserção à esquerda.
 - [2 pontos] **hash_table_items** para retornar um vector contendo todos os pares chave-valor (HashTableItem) existentes na tabela hash.
 - [2 pontos] **hash_table_count_collisions** para contar o número de colisões em cada posição da tabela hash. A função deve retornar um *array* de inteiros do mesmo tamanho da tabela hash contendo o número de colisões por posição.

O arquivo principal, main.c, realiza uma série de inserções/atualizações e, em seguida, mostra os elementos e o número de colisões por posição na tela. Chaves são o nome de uma pessoa e valores são números de telefone das pessoas.

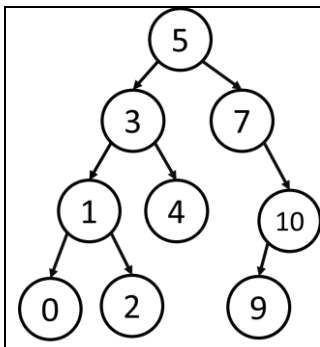
Restrição: É dada uma implementação de lista encadeada que não deve ser modificada. Assuma que a implementação da tabela hash pode usar o tipo Node da lista encadeada, isto é, ela não precisa ser tratada como uma estrutura opaca.

- 2) [5 pontos] Considere o código-fonte parcial de uma árvore binária de busca disponibilizado em anexo. Esta árvore armazena apenas números inteiros, ao invés de pares chave-valor. A sua tarefa é implementar a função **binary_tree_knn** que **busca dos k-vizinhos mais próximos à um valor dentro de um intervalo**. Dado um número inteiro **x**, um número inteiro **k** e um intervalo [**min**, **max**], a função deve encontrar os **k** elementos da árvore que pertençam a [**min**, **max**] e que tenham a menor diferença absoluta em relação a **x** (-1 e 5 possuem distância absoluta de 3 para o valor 2; a função **abs** pode ser usada para obter o valor absoluto). A função deve retornar um *array* de inteiros contendo os vizinhos mais próximos ordenados de forma ascendente e o número de elementos no *array*.

Se o número de elementos dentro do intervalo for menor que **k**, retorne apenas estes elementos. Se a árvore possuir menos de **k** elementos, retorne apenas os elementos da árvore.

Restrição: Não é válido colocar todos os elementos da árvore em um *array* e depois identificar os **k** vizinhos mais próximos dentro do intervalo iterando sobre o *array*. Deve ser feito um percurso na árvore e, para cada elemento:

- Se ele estiver fora do intervalo, ignore-o e siga para o próximo.
- Se o *array* de saída possuir menos de **k** elementos, adicione o valor no *array*.
- Caso contrário, se o valor do nó for mais próximo de **x** que algum dos **k** elementos do *array*, descarte o mais distante e adicione o novo valor de forma ordenada.



Exemplo 1: $X = 8$, $K = 4$, $\text{MIN} = 0$, $\text{MAX} = 20$

Resposta: [5, 7, 9, 10]

Exemplo 2: $X = 6$, $K = 3$, $\text{MIN} = 5$, $\text{MAX} = 20$

Resposta: [5, 7, 9]

Exemplo 3: $X = 8$, $K = 5$, $\text{MIN} = 2$, $\text{MAX} = 4$

Resposta: [2, 3, 4]